

```

"""AI-driven web scraper using ScrapeGraphAI (Local and SaaS)."""

from __future__ import annotations

import json
import logging
import os
import random as _random
import re
import sys
import time
from pathlib import Path

# Ensure project root is on sys.path so config imports work
PROJECT_ROOT = Path(__file__).resolve().parent.parent
sys.path.insert(0, str(PROJECT_ROOT))
from concurrent.futures import ThreadPoolExecutor, as_completed
from datetime import datetime, timedelta, timezone
from typing import Any, Optional

import requests
from requests.adapters import HTTPAdapter
from urllib3.util.retry import Retry

try:
    from scrapegraphai.graphs import SearchGraph, SmartScraperGraph
except ImportError:
    SmartScraperGraph = None
    SearchGraph = None

try:
    from scrapegraph_py import ScrapeGraphAI as ScrapeGraphSaaS
except ImportError:
    ScrapeGraphSaaS = None

from config.loader import load_config

logger = logging.getLogger(__name__)

# Suppress urllib3 retry noise from failing providers (e.g. Groq SSL errors)
logging.getLogger("urllib3.connectionpool").setLevel(logging.ERROR)

def _get_ai_config() -> Optional[dict]:
    """Retrieve ScrapeGraphAI configuration from config.yaml."""
    try:
        full_cfg = load_config()
        cfg = full_cfg.get("data_sources", {}).get("scrapegraphai", {})
        if not cfg.get("enabled"):
            return None

        # Local model config
        api_key = cfg.get("api_key")
        if (
            isinstance(api_key, str)
            and api_key.startswith("${")
            and api_key.endswith("}")
        ):
    
```

```

        api_key = os.getenv(api_key[2:-1])
if api_key:
    api_key = api_key.strip().strip('"').strip('\'')

# SaaS config
saas_api_key = cfg.get("saas_api_key")
if (
    isinstance(saas_api_key, str)
    and saas_api_key.startswith("${")
    and saas_api_key.endswith("}")
):
    saas_api_key = os.getenv(saas_api_key[2:-1])
if saas_api_key:
    saas_api_key = saas_api_key.strip().strip('"').strip('\'')

groq_api_key = cfg.get("groq_api_key")
if (
    isinstance(groq_api_key, str)
    and groq_api_key.startswith("${")
    and groq_api_key.endswith("}")
):
    groq_api_key = os.getenv(groq_api_key[2:-1])
if groq_api_key:
    groq_api_key = groq_api_key.strip().strip('"').strip('\'')

openrouter_api_key = cfg.get("openrouter_api_key")
if (
    isinstance(openrouter_api_key, str)
    and openrouter_api_key.startswith("${")
    and openrouter_api_key.endswith("}")
):
    openrouter_api_key = os.getenv(openrouter_api_key[2:-1])
if openrouter_api_key:
    openrouter_api_key =
openrouter_api_key.strip().strip('"').strip('\'')

sambanova_api_key = cfg.get("sambanova_api_key")
if (
    isinstance(sambanova_api_key, str)
    and sambanova_api_key.startswith("${")
    and sambanova_api_key.endswith("}")
):
    sambanova_api_key = os.getenv(sambanova_api_key[2:-1])
if sambanova_api_key:
    sambanova_api_key =
sambanova_api_key.strip().strip('"').strip('\'')

opencode_api_key = cfg.get("opencode_api_key")
if (
    isinstance(opencode_api_key, str)
    and opencode_api_key.startswith("${")
    and opencode_api_key.endswith("}")
):
    opencode_api_key = os.getenv(opencode_api_key[2:-1])
if opencode_api_key:
    opencode_api_key =
opencode_api_key.strip().strip('"').strip('\'')

```

```

        # Self-healing Fallback:
        # If the user accidentally set GOOGLE_API_KEY to their
ScrapeGraphAI key (starts with 'sgai-')
        # we route it to saas_api_key and clear api_key so the local
Gemini LLM doesn't get initialized with it.
        if isinstance(api_key, str) and api_key.startswith("sgai-"):
            if not saas_api_key:
                saas_api_key = api_key
            api_key = None

model_tokens = cfg.get("model_tokens")
llm_config = {
    "api_key": api_key,
    "model": cfg.get("model", "google_genai/gemini-1.5-flash"),
}
if model_tokens is not None:
    llm_config["model_tokens"] = model_tokens

return {
    "local": {
        "llm": llm_config,
        "verbose": False,
        "headless": True,
    }
    if api_key
    else None,
    "saas_api_key": saas_api_key,
    "groq_api_key": groq_api_key,
    "openrouter_api_key": openrouter_api_key,
    "sambanova_api_key": sambanova_api_key,
    "opencode_api_key": opencode_api_key,
}
except Exception as e:
    logger.error(f"Failed to load ScrapeGraphAI config: {e}")
    return None

# In-memory cache: skip dead providers for 600s after failure
_dead_providers: dict[str, float] = {}
_DEAD_PROVIDER_TTL = 600.0 # Re-try dead provider after 10 minutes

# Global rate limiter for SambaNova: max 1 call per 30 seconds to avoid
429
_sambanova_last_call_ts: float = 0.0
_SAMBANOVA_MIN_INTERVAL = 30.0 # seconds between SambaNova calls

# Global rate limiter for OpenRouter: max 1 call per 15 seconds to avoid
429
_openrouter_last_call_ts: float = 0.0
_OPENROUTER_MIN_INTERVAL = 15.0 # seconds between OpenRouter calls

# Global rate limiter for OpenCode Zen: max 1 call per 5 seconds
(generous free tier)
_opencode_zen_last_call_ts: float = 0.0
_OPENCODE_ZEN_MIN_INTERVAL = 5.0 # seconds between OpenCode Zen calls

def _sambanova_rate_limit_wait() -> None:

```

```
    """Enforce minimum interval between SambaNova API calls to prevent
429 rate limits.
```

```
    SambaNova free tier allows ~1 request per minute. We enforce a 30s
minimum
```

```
    interval with random jitter to avoid thundering herd.
    """
    global _sambanova_last_call_ts
    now = time.time()
    elapsed = now - _sambanova_last_call_ts
    if elapsed < _SAMBANOVA_MIN_INTERVAL:
        wait_time = (_SAMBANOVA_MIN_INTERVAL - elapsed) +
_random.uniform(0.5, 2.0)
        logger.debug(
            "SambaNova rate limiter: waiting %.1fs (elapsed=%.1fs)",
            wait_time, elapsed,
        )
        time.sleep(wait_time)
        _sambanova_last_call_ts = time.time()
```

```
def _opencode_zen_rate_limit_wait() -> None:
    """Enforce minimum interval between OpenCode Zen API calls."""
    global _opencode_zen_last_call_ts
    now = time.time()
    elapsed = now - _opencode_zen_last_call_ts
    if elapsed < _OPENCODE_ZEN_MIN_INTERVAL:
        wait_time = (_OPENCODE_ZEN_MIN_INTERVAL - elapsed) +
_random.uniform(0.3, 1.0)
        logger.debug(
            "OpenCode Zen rate limiter: waiting %.1fs (elapsed=%.1fs)",
            wait_time, elapsed,
        )
        time.sleep(wait_time)
        _opencode_zen_last_call_ts = time.time()
```

```
def _openrouter_rate_limit_wait() -> None:
    """Enforce minimum interval between OpenRouter API calls to prevent
429 rate limits.
```

```
    OpenRouter free tier has rate limits. We enforce a 15s minimum
    interval with random jitter to avoid thundering herd.
    """
    global _openrouter_last_call_ts
    now = time.time()
    elapsed = now - _openrouter_last_call_ts
    if elapsed < _OPENROUTER_MIN_INTERVAL:
        wait_time = (_OPENROUTER_MIN_INTERVAL - elapsed) +
_random.uniform(0.5, 2.0)
        logger.debug(
            "OpenRouter rate limiter: waiting %.1fs (elapsed=%.1fs)",
            wait_time, elapsed,
        )
        time.sleep(wait_time)
        _openrouter_last_call_ts = time.time()
```

```

class TLS12Adapter(HTTPAdapter):
    """HTTP Adapter that forces TLS 1.2 to bypass Windows OpenSSL/urllib3
    TLS 1.3 handshake bugs."""

    def init_poolmanager(self, *args, **kwargs):
        import ssl

        context = ssl.create_default_context()
        context.minimum_version = ssl.TLSVersion.TLSv1_2
        context.maximum_version = ssl.TLSVersion.TLSv1_2
        kwargs["ssl_context"] = context
        return super().init_poolmanager(*args, **kwargs)

def _create_curl_cffi_llm_session():
    """Create an LLM session using curl_cffi for robust SSL on Windows.

    curl_cffi bundles its own libcurl + cert bundle, bypassing the buggy
    Windows OpenSSL 3.x stack that causes SSLEOFError on some providers.
    Falls back to the TLS12Adapter approach if curl_cffi is not
    installed.

    Returns:
        tuple (session, backend_name) where backend_name is "curl_cffi"
    or "requests"
    """
    try:
        from curl_cffi import requests as curl_requests

        session = curl_requests.Session(impersonate="chrome120")
        return session, "curl_cffi"
    except ImportError:
        logger.debug("curl_cffi not available; using TLS12Adapter for LLM
        session")
        return _create_llm_retry_session(), "requests"

def _create_llm_retry_session(
    retries=1, backoff_factor=0.5, status_forcelist=(429, 500, 502, 503,
    504)
):
    """Creates a requests session with retry logic for LLM API calls.

    Reduced retries to save time/tokens on failing providers.
    SSL errors are not retried (immediate failover).
    """
    session = requests.Session()
    retry = Retry(
        total=retries,
        read=retries,
        connect=retries,
        backoff_factor=backoff_factor,
        status_forcelist=status_forcelist,
        allowed_methods=["HEAD", "GET", "OPTIONS", "POST"],
    )
    adapter = TLS12Adapter(max_retries=retry)
    session.mount("http://", adapter)
    session.mount("https://", adapter)

```

```

    return session

def _mark_provider_dead(provider: str, ttl: float = _DEAD_PROVIDER_TTL) -
> None:
    _dead_providers[provider] = time.time() + ttl

def _is_ssl_transport_error(exc: Exception) -> bool:
    text = repr(exc).lower()
    return (
        "ssl" in text or "unexpected_eof_while_reading" in text or
        "ssleoferror" in text
    )

def _is_http_error(exc: Exception) -> bool:
    """Check if exception is an HTTP error from requests or curl_cffi.

    curl_cffi.requests has its own HTTPError class that is NOT a subclass
    of
    requests.exceptions.HTTPError, so we check by class name.
    """
    cls_name = type(exc).__name__
    return cls_name == "HTTPError" or "HTTPError" in str(type(exc))

def _get_http_status(exc: Exception) -> int | None:
    """Extract HTTP status code from requests or curl_cffi HTTPError."""
    if hasattr(exc, "response") and exc.response is not None:
        try:
            return exc.response.status_code
        except Exception:
            pass
    return None

def _get_http_detail(exc: Exception, max_len: int = 200) -> str:
    """Extract response body from requests or curl_cffi HTTPError."""
    if hasattr(exc, "response") and exc.response is not None:
        try:
            text = exc.response.text
            if text:
                return text[:max_len]
        except Exception:
            pass
    return str(exc)[:max_len]

def _call_chat_completion(
    *,
    session: requests.Session | Any,
    url: str,
    headers: dict[str, str],
    model: str,
    system_prompt: str,
    prompt: str,
    json_mode: bool,

```

```

        timeout: int = 15,
) -> str:
    """Call a chat completion endpoint.

    Accepts both requests.Session and curl_cffi.requests.Session
    (they share the same .post() signature).
    """
    payload = {
        "model": model,
        "messages": [
            {"role": "system", "content": system_prompt},
            {"role": "user", "content": prompt},
        ],
        "response_format": {"type": "json_object"} if json_mode else
None,
    }
    resp = session.post(url, headers=headers, json=payload,
timeout=timeout)
    resp.raise_for_status()
    return resp.json()["choices"][0]["message"]["content"]

def call_llm(
    prompt: str,
    system_prompt: str = "You are a professional Indian stock market
strategist.",
    json_mode: bool = True,
    max_tokens: int | None = None,
    return_provider: bool = False,
) -> Any:
    """Universal LLM call with fallback: OpenCode Zen -> Groq."""
    config = _get_ai_config()
    if not config:
        return (None, "None") if return_provider else None

    # 1. OpenCode Zen (Primary) – only working model: big-pickle
    if config.get("opencode_api_key"):
        _opencode_zen_rate_limit_wait()
        session, backend = _create_curl_cffi_llm_session()
        logger.info("LLM #1: big-pickle [provider=OpenCode Zen]
(backend=%s)", backend)
        try:
            url = "https://opencode.ai/zen/v1/chat/completions"
            headers = {
                "Authorization": f"Bearer {config['opencode_api_key']}",
                "Content-Type": "application/json",
            }
            payload = {
                "model": "big-pickle",
                "messages": [
                    {"role": "system", "content": system_prompt},
                    {"role": "user", "content": prompt},
                ],
                "response_format": {"type": "json_object"} if json_mode
else None,
                "max_tokens": max_tokens,
            }

```

```

resp = session.post(url, headers=headers, json=payload,
timeout=20)
resp.raise_for_status()

if not resp.content or not resp.content.strip():
    raise ValueError("empty response body")
resp_data = resp.json()

if "error" in resp_data:
    err_msg = resp_data["error"]
    if isinstance(err_msg, dict):
        err_msg = err_msg.get("message", str(err_msg))
    logger.warning(
        "OpenCode Zen upstream error: %s. Trying Groq.",
err_msg,
    )
else:
    text = resp_data["choices"][0]["message"]["content"]
    if not text or not text.strip():
        raise ValueError("OpenCode Zen returned empty
content")

    res_val = json.loads(text) if json_mode else text
    logger.info("LLM success: big-pickle [OpenCode Zen]
(backend=%s)", backend)
    return (
        (res_val, "OpenCode Zen (big-pickle)")
        if return_provider
        else res_val
    )
except Exception as e:
    if _is_ssl_transport_error(e):
        logger.warning(
            "OpenCode Zen SSL error with %s backend: %s. Trying
Groq.",
            backend, e,
        )
    elif _is_http_error(e):
        status = _get_http_status(e)
        detail = _get_http_detail(e)
        if status == 401:
            logger.warning(
                "OpenCode Zen returned 401 (unauthorized). Check
OPENCODE_API_KEY. Trying Groq."
            )
        elif status == 429:
            logger.warning(
                "OpenCode Zen rate limited (429). Marking dead
for %ds. Trying Groq.",
                _DEAD_PROVIDER_TTL,
            )
            _mark_provider_dead("opencode_zen:big-pickle",
            _DEAD_PROVIDER_TTL)
        else:
            logger.warning(
                "OpenCode Zen HTTP error (%s): %s. Trying Groq.",
                status, detail,
            )

```

```

        elif isinstance(e, (json.JSONDecodeError, KeyError,
IndexError, ValueError)):
            logger.warning("OpenCode Zen call failed: %s. Trying
Groq.", e)
        else:
            logger.warning("OpenCode Zen call failed: %s. Trying
Groq.", e)

    # 2. Groq (Fallback)
    groq_dead_until = _dead_providers.get("groq")
    if groq_dead_until and time.time() < groq_dead_until:
        logger.debug("Groq marked dead until %.0f; skipping",
groq_dead_until)
    elif config.get("groq_api_key"):
        logger.info("LLM #2: llama-3.3-70b-versatile [provider=Groq]")
        session, backend = _create_curl_cffi_llm_session()
        try:
            url = "https://api.groq.com/openai/v1/chat/completions"
            headers = {
                "Authorization": f"Bearer {config['groq_api_key']}",
                "Content-Type": "application/json",
            }
            payload = {
                "model": "llama-3.3-70b-versatile",
                "messages": [
                    {"role": "system", "content": system_prompt},
                    {"role": "user", "content": prompt},
                ],
                "response_format": {"type": "json_object"} if json_mode
else None,
                "max_tokens": max_tokens,
            }
            resp = session.post(url, headers=headers, json=payload,
timeout=15)
            resp.raise_for_status()
            text = resp.json()["choices"][0]["message"]["content"]
            res_val = json.loads(text) if json_mode else text
            logger.info(
                "LLM success: llama-3.3-70b-versatile [Groq]
(backend=%s)", backend
            )
            return (
                (res_val, "Groq (llama-3.3-70b-versatile)")
                if return_provider
                else res_val
            )
        except Exception as e:
            # Handle SSL errors (mark Groq dead for a while)
            if _is_ssl_transport_error(e):
                _dead_providers["groq"] = time.time() +
_DEAD_PROVIDER_TTL
            logger.warning(
                "Groq SSL error with %s backend: %s. Marking dead for
%s. Trying Gemini.",
                backend,
                e,
                _DEAD_PROVIDER_TTL,
            )

```

```

        # Handle HTTP errors (works for both requests and curl_cffi)
        elif _is_http_error(e):
            status = _get_http_status(e)
            if status == 401:
                logger.warning(
                    "Groq returned 401 (unauthorized). Check
GROQ_API_KEY. Trying Gemini."
                )
            else:
                logger.warning(
                    "Groq HTTP error (%s): %s. Trying Gemini.",
                    status,
                    _get_http_detail(e),
                )

        # Handle JSON/parse errors
        elif isinstance(e, json.JSONDecodeError):
            logger.warning("Groq returned invalid JSON: %s. Trying
Gemini.", e)

        # Catch-all
        else:
            logger.warning("Groq call failed: %s. Trying Gemini.", e)

    # 3. Gemini (Fallback 2)
    if config["local"] and config["local"]["llm"].get("api_key"):
        session, backend = _create_curl_cffi_llm_session()
        try:
            api_key = config["local"]["llm"]["api_key"]
            model_raw = config["local"]["llm"].get("model", "gemini-1.5-
flash")
            model_name = model_raw.split("/")[-1] if "/" in model_raw
        else model_raw
        logger.info(
            "LLM #3: %s [provider=Gemini] (backend=%s)", model_name,
            backend
        )
        url =
f"https://generativelanguage.googleapis.com/v1beta/models/{model_name}:ge
nerateContent?key={api_key}"
        generation_config = {"response_mime_type":
"application/json"}
        if max_tokens is not None:
            generation_config["maxOutputTokens"] = max_tokens
        payload = {
            "contents": [
                {
                    "role": "user",
                    "parts": [
                        {"text": f"System: {system_prompt}\n\nUser:
{prompt}"}
                    ],
                }
            ],
            "generationConfig": generation_config if json_mode else
        },
    }

```

```

        resp = session.post(url, json=payload, timeout=15)
        resp.raise_for_status()
        res = resp.json()

        candidates = res.get("candidates", [])
        if not candidates:
            logger.warning(
                "Gemini returned no candidates (Safety filter?).
Response: %s", res
            )
            raise ValueError("No candidates in Gemini response")

        text = (
            candidates[0].get("content", {}).get("parts",
[{}])[0].get("text", "")
        )
        if not text:
            raise ValueError("Empty text part in Gemini response")

        res_val = json.loads(text) if json_mode else text
        logger.info("LLM success: %s [Gemini] (backend=%s)",
model_name, backend)
        return (res_val, f"Gemini ({model_name})") if return_provider
else res_val
    except Exception as e:
        if _is_ssl_transport_error(e):
            logger.warning(
                "Gemini SSL error with %s backend: %s. All providers
exhausted.",
                backend, e,
            )
        elif _is_http_error(e):
            status = _get_http_status(e)
            if status == 401:
                logger.warning(
                    "Gemini returned 401 (unauthorized). Check
GOOGLE_API_KEY. All providers exhausted."
                )
            else:
                logger.warning(
                    "Gemini HTTP error (%s) with %s backend: %s. All
providers exhausted.",
                    status, backend, _get_http_detail(e),
                )
        elif isinstance(e, json.JSONDecodeError):
            logger.warning("Gemini returned invalid JSON: %s. All
providers exhausted.", e)
        else:
            logger.warning(
                "Gemini call failed unexpectedly: %s. All providers
exhausted.", e,
            )

        logger.warning("LLM scan exhausted: all providers failed")
        return (None, "None") if return_provider else None

def test_llm_providers():

```

```

    """Diagnostic tool to verify LLM connectivity for all configured
    providers."""
    config = _get_ai_config()
    if not config:
        print("✗ Configuration not found. Check your config.yaml and
        enabled state.")
        return

    print("DEBUG CONFIG OPENCODE KEY:",
    repr(config.get("opencode_api_key")))
    print("DEBUG CONFIG SAMBANOVA KEY:",
    repr(config.get("sambanova_api_key")))
    print("DEBUG CONFIG GROQ KEY:", repr(config.get("groq_api_key")))
    print("DEBUG CONFIG OPENROUTER KEY:",
    repr(config.get("openrouter_api_key")))

    test_prompt = "Say 'Integration Successful'"
    results = {}

    def mask_key(k):
        if not k:
            return "None"
        if len(k) < 8:
            return "****"
        return f"{k[:4]}...{k[-4:]}"

    # 1. Test OpenCode Zen
    if config.get("opencode_api_key"):
        try:
            print(
                f"Testing OpenCode Zen (Key:
            {mask_key(config['opencode_api_key'])})..."
            )
            url = "https://opencode.ai/zen/v1/chat/completions"
            headers = {
                "Authorization": f"Bearer {config['opencode_api_key']}",
                "Content-Type": "application/json",
            }
            payload = {
                "model": "big-pickle",
                "messages": [{"role": "user", "content": test_prompt}],
                "max_tokens": 10,
            }
            resp = requests.post(url, headers=headers, json=payload,
            timeout=15)
            print("OpenCode Zen Raw Status:", resp.status_code)
            print("OpenCode Zen Raw Text:", resp.text)
            resp.raise_for_status()
            results["OpenCode Zen"] = "OK"
        except Exception as e:
            results["OpenCode Zen"] = f"Error: {type(e).__name__}:
            {str(e)}"
        else:
            results["OpenCode Zen"] = "Not Configured"

    # 2. Test SambaNova
    if config.get("sambanova_api_key"):
        try:

```

```

        print(
            f"Testing SambaNova (Key:
{mask_key(config['sambanova_api_key'])})..."
        )
        url = "https://api.sambanova.ai/v1/chat/completions"
        headers = {
            "Authorization": f"Bearer {config['sambanova_api_key']}",
            "Content-Type": "application/json",
        }
        payload = {
            "model": "DeepSeek-V3.1",
            "messages": [{"role": "user", "content": test_prompt}],
            "max_tokens": 10,
        }
        resp = requests.post(url, headers=headers, json=payload,
timeout=15)
        print("SambaNova Raw Status:", resp.status_code)
        print("SambaNova Raw Text:", resp.text)
        resp.raise_for_status()
        results["SambaNova"] = "OK"
    except Exception as e:
        results["SambaNova"] = f"Error: {type(e).__name__}: {str(e)}"
    else:
        results["SambaNova"] = "Not Configured"

# 3. Test Groq
if config.get("groq_api_key"):
    try:
        print(f"Testing Groq (Key:
{mask_key(config['groq_api_key'])})..."
        url = "https://api.groq.com/openai/v1/chat/completions"
        headers = {
            "Authorization": f"Bearer {config['groq_api_key']}",
            "Content-Type": "application/json",
        }
        payload = {
            "model": "llama-3.3-70b-versatile",
            "messages": [{"role": "user", "content": test_prompt}],
            "max_tokens": 10,
        }
        resp = requests.post(url, headers=headers, json=payload,
timeout=10)
        print("Groq Raw Status:", resp.status_code)
        print("Groq Raw Text:", resp.text)
        resp.raise_for_status()
        results["Groq"] = "OK"
    except Exception as e:
        results["Groq"] = f"Error: {type(e).__name__}: {str(e)}"
    else:
        results["Groq"] = "Not Configured"

# 4. Test Gemini
gemini_key = config["local"]["llm"].get("api_key") if config["local"]
else None
if gemini_key:
    try:
        print(f"Testing Gemini (Key: {mask_key(gemini_key)})...")
        res = call_llm(test_prompt, json_mode=False)

```

```

        results["Gemini"] = "OK" if res else "Empty Response"
    except Exception as e:
        err_msg = str(e)
        if "401" in err_msg:
            err_msg = "401 Unauthorized (Check your GOOGLE_API_KEY)"
        results["Gemini"] = f"Error: {err_msg}"
    else:
        results["Gemini"] = "Not Configured"

# 5. Test OpenRouter
if config.get("openrouter_api_key"):
    try:
        print(
            f"Testing OpenRouter (Key:
{mask_key(config['openrouter_api_key'])})..."
        )
        url = "https://openrouter.ai/api/v1/chat/completions"
        headers = {
            "Authorization": f"Bearer
{config['openrouter_api_key']}",
            "HTTP-Referer": "http://localhost:5050",
            "X-Title": "StockMinded",
            "Content-Type": "application/json",
        }
        payload = {
            "model": "meta-llama/llama-3.3-70b-instruct:free",
            "messages": [{"role": "user", "content": test_prompt}],
            "max_tokens": 10,
        }
        resp = requests.post(url, headers=headers, json=payload,
timeout=10)
        print("OpenRouter Raw Status:", resp.status_code)
        print("OpenRouter Raw Text:", resp.text)
        resp.raise_for_status()
        results["OpenRouter"] = "OK"
    except Exception as e:
        results["OpenRouter"] = f"Error: {type(e).__name__}:
{str(e)}"
    else:
        results["OpenRouter"] = "Not Configured"

print("\n--- LLM Provider Status ---")
for provider, status in results.items():
    symbol = (
        "[OK]"
        if status == "OK"
        else ("[--]" if status == "Not Configured" else "[ERROR]")
    )
    print(f"{provider}: {symbol} {status}")
print("-----\n")

def _ensure_dict(data: Any) -> Optional[dict]:
    """Convert ScrapeGraphAI pydantic response to dict if needed.

    Handles pydantic v1 (BaseModel.dict), pydantic v2
(BaseModel.model_dump),
    and nested pydantic models recursively.

```

```

"""
if data is None:
    return None
if isinstance(data, dict):
    return data

# Pydantic v2: model_dump(mode="json") recursively converts all
fields
if hasattr(data, "model_dump"):
    try:
        return data.model_dump(mode="json")
    except Exception as e:
        logging.getLogger(__name__).debug(
            "[_ensure_dict] model_dump(json) failed: %s", e
        )
    try:
        return data.model_dump()
    except Exception as e:
        logging.getLogger(__name__).debug(
            "[_ensure_dict] model_dump() failed: %s", e
        )

# Pydantic v1: .dict()
if hasattr(data, "dict"):
    try:
        return data.dict()
    except Exception as e:
        logging.getLogger(__name__).debug("[_ensure_dict] .dict()
failed: %s", e)

# Try converting via __dict__ or vars()
if hasattr(data, "__dict__"):
    result = {}
    for k, v in data.__dict__.items():
        if not k.startswith("_"):
            result[k] = (
                _ensure_dict(v)
                if hasattr(v, "model_dump") or hasattr(v, "dict")
                else v
            )
    if result:
        return result

# Last resort: serialize/deserialize
try:
    return json.loads(json.dumps(data, default=str))
except Exception:
    logger.debug(f"Could not convert response to dict:
{type(data).__name__}")
    return None

def scrape_url(url: str, prompt: str) -> Optional[dict]:
    """Extract structured data from a single URL using SaaS (preferred)
or Local graph.

Returns:
    dict with extracted data, or None if extraction failed.

```

```

"""
config = _get_ai_config()
if not config:
    return None

# Try SaaS first if configured
if config["saas_api_key"] and ScrapeGraphSaaS:
    try:
        logger.info(f"AI SaaS Extract started for URL: {url}")
        sgai = ScrapeGraphSaaS(api_key=config["saas_api_key"])
        # In SaaS, extract() is used for structured data from URL
        result = sgai.extract(prompt=prompt, url=url)
        if result.status == "success":
            return _ensure_dict(result.data)
        else:
            logger.warning(f"SaaS Extract failed for {url}:
{result.error}")
    except Exception as e:
        logger.error(f"SaaS Extract failed for {url}: {e}")

# Fallback to local execution if library is installed
if config["local"] and SmartScraperGraph:
    try:
        logger.info(f"AI Local Scrape fallback started for URL:
{url}")
        graph = SmartScraperGraph(prompt=prompt, source=url,
config=config["local"])
        return _ensure_dict(graph.run())
    except Exception as e:
        logger.error(f"Local SmartScraperGraph failed for {url}:
{e}")

return None

def search_and_scrape(query: str, prompt: str) -> Any:
    """Search for information across multiple pages and summarize."""
    config = _get_ai_config()
    if not config:
        return None

    # Try SaaS first
    if config["saas_api_key"] and ScrapeGraphSaaS:
        try:
            logger.info(f"AI SaaS Search started for query: {query}")
            sgai = ScrapeGraphSaaS(api_key=config["saas_api_key"])
            # 1. Perform the search
            result = sgai.search(query=query)
            if result.status == "success":
                search_data = result.data
                results_list = []
                if isinstance(search_data, dict):
                    results_list = search_data.get("results") or []
                elif hasattr(search_data, "results"):
                    results_list = getattr(search_data, "results") or []
                elif isinstance(search_data, list):
                    results_list = search_data

```

```

        text_parts = []
        for item in results_list:
            title = ""
            content = ""
            if isinstance(item, dict):
                title = item.get("title") or ""
                content = item.get("content") or ""
            else:
                title = getattr(item, "title", "") or ""
                content = getattr(item, "content", "") or ""
            if title or content:
                text_parts.append(f"Title: {title}\nContent:
{content}\n")

        if text_parts:
            combined_text = "\n".join(text_parts)
            logger.info(
                f"AI SaaS Extract started for search results of
query: {query}"
            )
            summary_res = sgai.extract(prompt=prompt,
html=combined_text)
            if summary_res.status == "success":
                data = summary_res.data
                if hasattr(data, "json_data") and data.json_data:
                    return data.json_data
                if isinstance(data, dict) and "json_data" in
data:
                    return data["json_data"]
                return data
            else:
                logger.warning(
                    f"SaaS Extract for search results failed:
{summary_res.error}"
                )
            else:
                logger.warning("No search results found to extract.")
        else:
            logger.warning(
                f"SaaS Search failed for query '{query}':
{result.error}"
            )
        except Exception as e:
            logger.error(f"SaaS Search failed for query '{query}': {e}")

    return None

def get_option_chain_fallback(symbol: str) -> Optional[dict]:
    """Fallback extraction of option chain data using AI."""
    url = f"https://www.research360.in/future-and-options/option-
chain?symbol={symbol}"
    prompt = (
        "Extract the current option chain table. "
        "Return a JSON object with 'records' containing a list of 'data'
points. "
        "Each data point should have 'strikePrice', 'CE' (openInterest,
lastPrice), "

```

```

        "and 'PE' (openInterest, lastPrice). "
        "Also include 'underlyingValue' if found."
    )
    return scrape_url(url, prompt)

def get_fii_dii_fallback() -> Optional[list[dict]]:
    """Fallback extraction of FII/DII cash data using AI."""
    url =
    "https://www.moneycontrol.com/stocks/marketstats/fii_dii_activity/index.p
hp"
    prompt = (
        "Extract the latest FII and DII cash market net investment data.
"
        "Return a list of objects, each with 'category' (FII or DII),
'date', and 'netValue' in Crores."
    )
    return scrape_url(url, prompt)

def _get_persistent_sentiment_cache() -> tuple[Optional[Any], float,
float]:
    """Retrieve cached sentiment, timestamp, and expires_at from a
persistent local file."""
    import json
    from pathlib import Path

    cache_file = Path("data/cache/ai_sentiment_cache.json")
    if cache_file.exists():
        try:
            with open(cache_file, "r") as f:
                data = json.load(f)
                return (
                    data.get("sentiment"),
                    data.get("timestamp", 0.0),
                    data.get("expires_at", 0.0),
                )
        except Exception as e:
            logger.exception("Failed to read persistent sentiment cache:
%s", e)
    return None, 0.0, 0.0

def _set_persistent_sentiment_cache(
    sentiment: Any, timestamp: float, ttl: float = 3600.0
) -> None:
    """Save sentiment, timestamp, and expires_at to a persistent local
file."""
    import json
    from pathlib import Path

    cache_file = Path("data/cache/ai_sentiment_cache.json")
    try:
        cache_file.parent.mkdir(parents=True, exist_ok=True)
        with open(cache_file, "w") as f:
            json.dump(
                {
                    "sentiment": sentiment,

```

```

        "timestamp": timestamp,
        "expires_at": timestamp + ttl,
    },
    f,
)
except Exception as e:
    logger.exception("Failed to write persistent sentiment cache:
%s", e)

def _get_sentiment_history() -> list[dict]:
    """Load sentiment analysis history for self-improvement loop."""
    import json
    from pathlib import Path

    hist_file = Path("data/cache/ai_sentiment_history.json")
    if hist_file.exists():
        try:
            with open(hist_file, "r") as f:
                data = json.load(f)
                return data if isinstance(data, list) else []
        except Exception as e:
            logger.exception("Failed to read sentiment history: %s", e)
    return []

def _save_sentiment_run(run: dict) -> None:
    """Save a sentiment analysis run to history for self-improvement."""
    import json
    from pathlib import Path

    hist_file = Path("data/cache/ai_sentiment_history.json")
    try:
        hist_file.parent.mkdir(parents=True, exist_ok=True)
        history = _get_sentiment_history()
        history.append(run)
        # Keep last 20 runs to bound file size
        history = history[-20:]
        with open(hist_file, "w") as f:
            json.dump(history, f, indent=2)
    except Exception as e:
        logger.exception("Failed to save sentiment history: %s", e)

def _parse_rss_date(date_str: str) -> float:
    """Parse RFC 2822 date string to Unix timestamp. Returns 0.0 on
failure."""
    from email.utils import parsedate_to_datetime

    try:
        return parsedate_to_datetime(date_str).timestamp()
    except Exception:
        return 0.0

# Noise phrases – skip headlines that are generic market outlooks /
predictions
_NOISE_PATTERNS = {

```

```

    "prediction",
    "preview",
    "market today",
    "what to expect",
    "how indian",
    "stock market is expected",
    "nifty 50, sensex",
    "nifty 50, bank nifty",
    "nifty prediction",
    "banknifty prediction",
    "bank nifty prediction",
    "share market today",
    "sensex today",
    "outlook",
    "weekly recap",
    "week ahead",
    "market wrap",
    "roundup",
    "watch list",
    "stocks to watch",
    "things to know",
    "pre-market",
    "premarket",
    "gift nifty",
}

```

```

_LOW_VALUE_TOKENS = {
    "ai",
    "market",
    "markets",
    "data",
    "nifty",
    "banknifty",
    "india",
    "vix",
    "rupee",
    "stocks",
    "stock",
    "share",
    "shares",
    "today",
    "live",
    "news",
    "update",
    "highlights",
    "buzz",
    "watch",
    "preview",
}

```

```

def _is_noise_headline(title: str) -> bool:
    t = title.lower()
    return any(p in t for p in _NOISE_PATTERNS)

```

```

def _is_low_value_headline(title: str) -> bool:
    import re as _re

```

```

tokens = [t for t in _re.findall(r"[a-zA-Z&]+", title.lower()) if t]
if len(tokens) < 3:
    return True
low_value_hits = sum(1 for t in tokens if t in _LOW_VALUE_TOKENS)
if low_value_hits / max(len(tokens), 1) >= 0.6:
    return True
useful_tokens = [t for t in tokens if t not in _LOW_VALUE_TOKENS]
return len(useful_tokens) < max(1, len(tokens) // 3)

def _is_relevant_to_indian_market(title: str) -> bool:
    """Cheap relevance check to prevent sentiment pipeline crashing.

    This function is intentionally heuristic and fast.
    It should only be used to avoid sending clearly-irrelevant headlines
    into the LLM/analysis pipeline.
    """
    if not title:
        return False

    t = title.lower()

    # Strong signals for Indian-market context
    indian_markers = (
        "india",
        "indian",
        "nse",
        "bse",
        "sensex",
        "nifty",
        "banknifty",
        "rupee",
        "rbi",
        "sebi",
        "msci india",
        "foreign portfolio",
        "fii",
        "dii",
        "gsec",
    )
    if any(m in t for m in indian_markers):
        return True

    # Common ticker-ish patterns: uppercase tickers often appear inside
    headlines
    # (we keep it conservative: require at least one known market word +
    ticker token)
    ticker_like = bool(re.search(r"\b[A-Z]{2,5}\b", title))
    if ticker_like and any(
        x in t
        for x in (
            "shares",
            "stock",
            "company",
            "results",
            "earnings",
            "quarter",

```

```

        "order",
    )
):
    return True

# If it's explicitly a global macro without Indian cues, reject.
global_noise_markers = (
    "oil prices",
    "us stocks",
    "dow jones",
    "nasdaq",
    "s&p 500",
    "fed",
    "eurozone",
    "ftse",
    "dollar index",
    "china stocks",
)
if any(m in t for m in global_noise_markers):
    return False

# Default: accept if it mentions at least one Indian
regulatory/financial institution
fallback_markers = (
    "moneycontrol",
    "livemint",
    "icici",
    "hdfc",
    "axis",
    "sbi",
)
return any(m in t for m in fallback_markers)

def _normalize_headline(title: str) -> str:
    import re as _re

    cleaned = _re.sub(r"^[a-z0-9 ]", " ", title.lower())
    return " ".join(cleaned.split())

def _dedupe_headlines(headlines: list[tuple[str, str]]) ->
list[tuple[str, str]]:
    seen: set[str] = set()
    deduped: list[tuple[str, str]] = []
    for title, pub in headlines:
        key = _normalize_headline(title)
        if not key or key in seen:
            continue
        seen.add(key)
        deduped.append((title, pub))
    return deduped

# Redesigned to match the new structured JSON output
# Expected from Gemini for consistency.
def analyze_sentiment_locally(headlines: list[tuple[str, str]]) -> dict:
    """Analyze news sentiment locally using a curated financial lexicon.

```

```

Args:
    headlines: List of (title, pub_date_str) tuples.

Returns:
    Sentiment dict matching the Gemini output schema.
"""
# --- Curated lexicon: refined to be more precise.
# Removed noisy words (hit, high, up, down, support, sell, pressure)
that fire on neutral headlines.
# Added India-specific terms.
pos_words = {
    "rally",
    "rebound",
    "gain",
    "gains",
    "rise",
    "rises",
    "surge",
    "surges",
    "soar",
    "soars",
    "upbeat",
    "bull",
    "bullish",
    "optimistic",
    "optimism",
    "recovery",
    "breakout",
    "beat",
    "beats",
    "growth",
    "strong",
    "positive",
    "buying",
    "inflow",
    "stabilize",
    "stabilizing",
    "upgrade",
    "upgraded",
    "gaining",
    "soaring",
    "outperform",
    "record",
    "boom",
    "booming",
    "accumulate",
    "accumulation",
    "overweight",
    "strengthens",
    "support",
}
neg_words = {
    "drop",
    "drops",
    "fall",
    "falls",
    "plunge",

```

```

    "plunges",
    "crash",
    "decline",
    "declines",
    "selling",
    "downgrade",
    "downgraded",
    "loss",
    "losses",
    "slump",
    "slumps",
    "bear",
    "bearish",
    "fears",
    "worry",
    "worries",
    "correction",
    "slashed",
    "underperform",
    "weak",
    "weakness",
    "sink",
    "drag",
    "outflow",
    "slowdown",
    "falling",
    "plunging",
    "declining",
    "worried",
    "selloff",
    "panic",
    "turmoil",
    "underweight",
    "weakens",
    "resistance",
}
# Phrase scoring for higher-signal events
phrase_scores = {
    "rate cut": 2,
    "rate cuts": 2,
    "rate hike": -2,
    "rate hikes": -2,
    "earnings beat": 2,
    "beats estimates": 2,
    "misses estimates": -2,
    "earnings miss": -2,
    "profit warning": -2,
    "order win": 2,
    "order book": 1,
    "stake sale": -1,
    "buyback": 2,
    "dividend": 1,
    "fund raise": 1,
    "regulatory action": -2,
    "sebi action": -2,
    "rbi policy": 1,
    "inflation rises": -2,
    "inflation cools": 2,

```

```

        "rupee weakens": -1, # Already covered by 'weakens' but good for
explicit phrase
        "rupee strengthens": 1,
        "fii buying": 2,
        "fii selling": -2,
    }

    # Simple negation prefixes that flip the next word's polarity
    negators = {"no", "not", "fails", "failed", "unlikely", "without",
"never"}

    pos_score = 0
    neg_score = 0 # Total positive/negative signals detected
    catalysts: list[str] = []
    # Accumulate per-ticker net sentiment: {ticker: int} (+ = bullish
mentions, - = bearish)
    ticker_sentiment: dict[str, int] = {}

    # Major Indian stock tickers - includes common headline name variants
    known_tickers = {
        "RELIANCE",
        "TCS",
        "HDFCBANK",
        "INFY",
        "ICICIBANK",
        "SBIN",
        "BHARTIARTL",
        "LT",
        "ITC",
        "TATASTEEL",
        "TATAMOTORS",
        "MARUTI",
        "AXISBANK",
        "KOTAKBANK",
        "ADANIENT",
        "WIPRO",
        "HCLTECH",
        "SUNPHARMA",
        "BAJFINANCE",
        "COALINDIA",
        "NIFTY",
        "BANKNIFTY",
    }

    # Tickers that need substring match because they contain special
characters
    # or appear as company names rather than ticker symbols
    substring_tickers = {"M&M": "M&M", "MAHINDRA": "M&M"}

    headline_catalysts: list[dict] = [] # To store structured catalysts

    for title, _pub_date in headlines:
        lower_title = title.lower()
        phrase_pos = 0
        phrase_neg = 0
        for phrase, score in phrase_scores.items():
            if phrase in lower_title:
                if score > 0:
                    phrase_pos += score

```

```

        elif score < 0:
            phrase_neg += abs(score)
words = [w.strip(".",;:?!()\\"'") for w in lower_title.split()]

item_pos = 0
item_neg = 0
for i, w in enumerate(words):
    prev_word = words[i - 1] if i > 0 else ""
    is_negated = prev_word in negators

    if w in pos_words:
        if is_negated:
            item_neg += 1
        else:
            item_pos += 1
    elif w in neg_words:
        if is_negated:
            item_pos += 1
        else:
            item_neg += 1

item_pos += phrase_pos
item_neg += phrase_neg
pos_score += item_pos
neg_score += item_neg # Accumulate total signals

# Extract structured catalysts
if not _is_noise_headline(title):
    clean_title = title.split(" - ")[0].strip()
    if item_pos > item_neg:
        headline_catalysts.append(
            {"type": "POSITIVE", "description": clean_title}
        )
    elif item_neg > item_pos:
        headline_catalysts.append(
            {"type": "NEGATIVE", "description": clean_title}
        )
    else:
        headline_catalysts.append(
            {"type": "NEUTRAL", "description": clean_title}
        )

# Limit catalysts to top 3 by sentiment strength
headline_catalysts.sort(key=lambda x: abs(item_pos - item_neg),
reverse=True)
catalysts = headline_catalysts[:3]

# Detect tickers - accumulate net sentiment per ticker
headline_direction = (
    1 if item_pos > item_neg else (-1 if item_neg > item_pos else
0)
)
upper_title = title.upper()
title_words_upper = {w.strip(".",;:?!()\\"'") for w in
upper_title.split()}

for ticker in known_tickers:
    if ticker in title_words_upper and headline_direction != 0:

```

```

        ticker_sentiment[ticker] = (
            ticker_sentiment.get(ticker, 0) + headline_direction
        )
    for substr, canonical in substring_tickers.items():
        if substr in upper_title and headline_direction != 0:
            ticker_sentiment[canonical] = (
                ticker_sentiment.get(canonical, 0) +
headline_direction
            )

    # Build deduplicated trade ideas: one direction per ticker
    trade_ideas: list[dict] = []
    for ticker, net in sorted(
        ticker_sentiment.items(), key=lambda x: abs(x[1]), reverse=True
    ):
        if len(trade_ideas) >= 3:
            break
        if net == 0 or abs(net) < 2:
            continue
        side = "LONG" if net > 0 else "SHORT"
        mentions = abs(
            net
        ) # Number of times this ticker was mentioned with a directional
sentiment
        strength = (
            "strong" if mentions >= 3 else "moderate" if mentions == 2
else "mild"
        )
        trade_ideas.append(
            {
                "direction": side,
                "ticker": ticker,
                "reason": f"{strength} {side.lower()} bias from
{mentions} news mentions.",
            }
        )

    # Fallback catalysts: only use non-noisy headlines
    if not catalysts:
        fallback_titles = [
            title.split(" - ")[0].strip()
            for title, _ in headlines
            if title and not _is_noise_headline(title)
       ][:3]
        catalysts = [{"type": "NEUTRAL", "description": t} for t in
fallback_titles]

    # Final fallback if all headlines were noise
    if not catalysts:
        if pos_score > neg_score:
            catalysts = [
                {
                    "type": "POSITIVE",
                    "description": "Broad market sentiment positive
across recent news cycle.",
                }
            ]
        elif neg_score > pos_score:

```

```

        catalysts = [
            {
                "type": "NEGATIVE",
                "description": "Caution signals detected across
recent news cycle.",
            }
        ]
    else:
        catalysts = [
            {
                "type": "NEUTRAL",
                "description": "Mixed signals – no dominant
directional catalyst identified.",
            }
        ]

```

```

# Determine overall sentiment and strength
total_signals = pos_score + neg_score

sentiment_score = 0.0
if total_signals > 0:
    sentiment_score = (pos_score - neg_score) / total_signals

# Define ratio before conditionals so it's always available
ratio = (pos_score / total_signals) if total_signals > 0 else 0.5

if sentiment_score > 0.3:
    sentiment = "BULLISH"
elif sentiment_score < -0.3:
    sentiment = "BEARISH"
else:
    sentiment = "NEUTRAL"

sentiment_strength = "WEAK"
if abs(sentiment_score) > 0.5 and total_signals >= 5:
    sentiment_strength = "STRONG"
elif abs(sentiment_score) > 0.2 and total_signals >= 3:
    sentiment_strength = "MODERATE"

confidence = "LOW"
if sentiment_strength == "MODERATE":
    confidence = "MEDIUM"
elif sentiment_strength == "STRONG":
    confidence = "HIGH"
if total_signals < 3:
    sentiment = "NEUTRAL"
    confidence = "LOW"

if total_signals < 3:
    justification = (
        f"Local Sentiment Engine: analyzed {len(headlines)} headlines
with low signal density. "
        f"{pos_score} bullish vs {neg_score} bearish signals (ratio
{ratio:.0%})."
    )
else:
    justification = (

```

```

        f"Local Sentiment Engine: analyzed {len(headlines)}
headlines. "
        f"{pos_score} bullish vs {neg_score} bearish signals "
        f"(ratio {ratio:.0%})."
    )

    return {
        "overall_market_sentiment": sentiment, # BULLISH, BEARISH,
NEUTRAL
        "sentiment_score": round(sentiment_score, 2), # -1.0 to 1.0
        "sentiment_strength": sentiment_strength, # WEAK, MODERATE,
STRONG
        "justification": justification,
        "key_catalysts": catalysts, # List of {type:
POSITIVE/NEGATIVE/NEUTRAL, description: str}
        "actionable_trade_ideas": trade_ideas, # List of {direction:
LONG/SHORT, ticker: str, reason: str}
        "confidence": confidence, # LOW, MEDIUM, HIGH
    }

```

```

def _fetch_and_parse_rss(query: str, recency_cutoff: float) ->
list[tuple[str, str]]:
    """Fetch and parse a single Google News RSS feed."""
    headlines = []
    try:
        rss_query = urllib.parse.quote(query)
        rss_url = (
            f"https://news.google.com/rss/search?q={rss_query}"
            "&hl=en-IN&gl=IN&ceid=IN:en"
        )
        resp = requests.get(rss_url, headers={"User-Agent":
"Mozilla/5.0"}, timeout=10)
        resp.raise_for_status()
        root = ET.fromstring(resp.content)

        for item in root.findall(".//item"):
            title_el = item.find("title")
            date_el = item.find("pubDate")
            title = (
                title_el.text.strip() if title_el is not None and
title_el.text else ""
            )
            pub_date = (
                date_el.text.strip() if date_el is not None and
date_el.text else ""
            )
            if not title or _is_noise_headline(title) or
_is_low_value_headline(title):
                continue
            if (
                pub_date
                and (pub_ts := _parse_rss_date(pub_date)) > 0
                and pub_ts < recency_cutoff
            ):
                continue
            headlines.append((title, pub_date))
            if len(headlines) >= 20:

```

```

        break
    except Exception as e:
        logger.warning(f"RSS fetch for query '{query}' failed: {e}")
    return headlines

def _fetch_icicidirect_news() -> list[tuple[str, str]]:
    """Fetch recent headlines from ICICI Direct market commentary."""
    url = "https://www.icicidirect.com/share-market-today/market-news-commentary"
    headlines = []
    try:
        logger.info("Fetching ICICI Direct news-flow...")
        try:
            from curl_cffi import requests as curl_requests

            session = curl_requests.Session(impersonate="chrome120")
            resp = session.get(url, timeout=20)
        except ImportError:
            headers = {
                "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/120.0.0.0 Safari/537.36",
            }
            resp = requests.get(url, headers=headers, timeout=20)

        resp.raise_for_status()

        # Extract headlines from h2-h4 tags which contain commentary
        titles = re.findall(
            r"<h[2-4][^>]*>(.*?)</h[2-4]>", resp.text, re.DOTALL | re.IGNORECASE
        )
        for title in titles[:15]:
            clean = re.sub(r"<[^>]+>", "", title).strip()
            if (
                clean
                and len(clean) > 15
                and not _is_noise_headline(clean)
                and not _is_low_value_headline(clean)
            ):
                headlines.append((clean,
datetime.now(timezone.utc).isoformat()))

        logger.info(
            f"Successfully extracted {len(headlines)} headlines from ICICI Direct"
        )
    except Exception as e:
        logger.warning(f"ICICI Direct news fetch failed: {e}")
    return headlines

def _fetch_livemint_news() -> list[tuple[str, str]]:
    """Fetch recent headlines from Livemint's market news section."""
    url = "https://www.livemint.com/market/stock-market-news"
    headlines = []
    try:

```

```

logger.info("Fetching Livemint news-flow...")
try:
    from curl_cffi import requests as curl_requests

    session = curl_requests.Session(impersonate="chrome120")
    resp = session.get(url, timeout=20)
except ImportError:
    resp = requests.get(url, headers={"User-Agent":
"Mozilla/5.0"}, timeout=20)

    resp.raise_for_status()
    # Match headlines inside h2 tags which are common for Livemint's
listing page
    titles = re.findall(
        r"<h2[^>]*>.*?<a[^>]*>(.*?)</a>", resp.text, re.DOTALL |
re.IGNORECASE
    )
    for title in titles[:15]:
        clean = re.sub(r"<[^>]+>", "", title).strip()
        if (
            clean
            and len(clean) > 20
            and not _is_noise_headline(clean)
            and not _is_low_value_headline(clean)
        ):
            headlines.append((clean,
datetime.now(timezone.utc).isoformat()))

    logger.info(f"Successfully extracted {len(headlines)} headlines
from Livemint")
except Exception as e:
    logger.warning(f"Livemint news fetch failed: {e}")
return headlines

def _fetch_moneycontrol_news() -> list[tuple[str, str]]:
    """Fetch recent headlines from Moneycontrol Business/Markets
section."""
    url = "https://www.moneycontrol.com/news/business/markets/"
    headlines = []
    try:
        logger.info("Fetching Moneycontrol news-flow...")
        try:
            from curl_cffi import requests as curl_requests

            session = curl_requests.Session(impersonate="chrome120")
            resp = session.get(url, timeout=20)
        except ImportError:
            headers = {
                "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/120.0.0.0 Safari/537.36",
            }
            resp = requests.get(url, headers=headers, timeout=20)

        resp.raise_for_status()

        # Moneycontrol headlines are typically in h2 tags within a
specific list structure

```

```

        # We look for the 'title' attribute or the inner text of anchors
within h2
        titles = re.findall(
            r'<h2[^>]*>.*?<a[^>]*title="([^\"]*)"', resp.text, re.DOTALL |
re.IGNORECASE
        )
        if not titles:
            titles = re.findall(
                r'<h2[^>]*>.*?<a[^>]*>(.*)</a>', resp.text, re.DOTALL |
re.IGNORECASE
            )

        for title in titles[:15]:
            clean = re.sub(r"<[^>]+>", "", title).strip()
            if (
                clean
                and len(clean) > 15
                and not _is_noise_headline(clean)
                and not _is_low_value_headline(clean)
            ):
                headlines.append((clean,
datetime.now(timezone.utc).isoformat()))

        logger.info(
            f"Successfully extracted {len(headlines)} headlines from
Moneycontrol"
        )
    except Exception as e:
        logger.warning(f"Moneycontrol news fetch failed: {e}")
    return headlines

def get_market_news_sentiment() -> Optional[dict]:
    """Fetch market news, summarize sentiment via LLM or local lexicon
    fallback.

    Pipeline:
    1. Fetch headlines from ICICI Direct, Livemint, Moneycontrol.
    2. Attempt LLM summarization (Groq → Gemini → OpenRouter).
    3. If LLM fails → local lexicon analysis (0 API calls).
    """
    import time

    now = time.time()

    # Check persistent cache
    cached_val, cached_ts, expires_at = _get_persistent_sentiment_cache()
    if expires_at == 0.0 and cached_ts > 0.0:
        expires_at = cached_ts + 3600.0

    if cached_val is not None and now < expires_at:
        logger.info(
            f"Returning cached sentiment (age: {round(now - cached_ts)}s,
"
            f"expires in {round(expires_at - now)}s)"
        )
    return cached_val

```

```

# 1. Fetch headlines from all sources in parallel
headlines: list[tuple[str, str]] = []
recency_cutoff = now - 36 * 3600 # Last 36 hours only

with ThreadPoolExecutor(max_workers=4) as executor:
    future_to_source = {
        executor.submit(_fetch_icicidirect_news): "icicidirect",
        executor.submit(_fetch_livemint_news): "livemint",
        executor.submit(_fetch_moneycontrol_news): "moneycontrol",
    }

    for future in as_completed(future_to_source):
        source_name = future_to_source[future]
        try:
            result = future.result()
            if result:
                headlines.extend(result)
                logger.info(
                    f"Source '{source_name}' contributed
{len(result)} headlines."
                )
        except Exception as exc:
            logger.error(f"Source '{source_name}' generated an
exception: {exc}")

    headlines = _dedupe_headlines(headlines)

    # Verification step: Filter for Indian market relevance
    original_count = len(headlines)
    headlines = [h for h in headlines if
_is_relevant_to_indian_market(h[0])]
    if original_count > 0 and len(headlines) < original_count:
        logger.info(
            f"Relevance filter: {len(headlines)}/{original_count}
headlines passed Indian market verification."
        )

    if not headlines:
        fallback = {
            "overall_market_sentiment": "NEUTRAL",
            "justification": "News feed returned no actionable headlines
in the last 36 hours.",
            "top_catalysts": ["Low-quality or duplicate headlines
filtered out."],
            "actionable_trade_ideas": [],
        }
        _set_persistent_sentiment_cache(fallback, now, ttl=900.0)
        return fallback

    logger.info(f"Fetched a total of {len(headlines)} unique headlines
for analysis.")

# 2. Unified LLM analysis (Gemini -> Groq -> OpenRouter)
logger.info("Performing news sentiment analysis via Brain Chain...")
news_text = "\n".join(f"- {pub}: {title}" for title, pub in
headlines)

```

```

    # Self-improvement: load past sentiment history so LLM can learn from
prior misses
    _sentiment_history = _get_sentiment_history()
    _history_context = ""
    if _sentiment_history:
        _recent = _sentiment_history[-3:]
        _history_context = (
            "\n\n--- PREVIOUS SENTIMENT ANALYSIS HISTORY (Self-
Improvement Memory) ---\n"
            "Review how your previous analyses performed. Learn from
prior misreads:\n"
        )
        for _h in _recent:
            _ts = _h.get("timestamp", "N/A")
            _s = _h.get("overall_market_sentiment", "N/A")
            _m = _h.get("model_used", "unknown")
            _history_context += f"- [{_ts}] Sentiment: {_s} (Model:
{_m})\n"
        _history_context += (
            "Use this memory to avoid repeating past errors. "
            "If a previous sentiment call was inaccurate, adjust your
weighting.\n"
        )

    prompt = (
        "Analyze these LATEST Indian stock market news headlines from the
last 12 hours. "
        "Your analysis MUST be based SOLELY on the provided headlines. "
        "Strictly ignore your internal training data about past years;
ONLY use the headlines provided. "
        "If no specific recent events are found in the text, do not
invent them. Return JSON:\n"
        "1. 'overall_market_sentiment': BULLISH/BEARISH/NEUTRAL\n"
        "2. 'sentiment_score': float -1 to 1\n"
        "3. 'sentiment_strength': WEAK/MODERATE/STRONG\n"
        "4. 'justification': 1-2 sentence summary based EXCLUSIVELY on
the provided headlines.\n"
        "5. 'key_catalysts': list of {type: POS/NEG/NEUT, description:
str}\n"
        "6. 'actionable_trade_ideas': list of {direction: LONG/SHORT,
ticker: str, reason: str}\n"
        "7. 'confidence': LOW/MEDIUM/HIGH\n"
        "8. 'model_used': (will be filled by system)\n\n"
        f"{_history_context}"
        f"Headlines:\n{news_text}"
    )

    sentiment, model_used = call_llm(prompt, return_provider=True)

# 3. Last fallback: Local lexicon
if not sentiment:
    logger.info("Brain Chain failed. Using Local Lexicon fallback.")
    try:
        sentiment = analyze_sentiment_locally(headlines)
        model_used = "Local Lexicon (fallback)"
    except Exception as lex_err:
        logger.error("Local lexicon fallback failed: %s", lex_err)
        sentiment = None

```

```

        model_used = "None"

    # Tag sentiment with model info if it's a dict
    if isinstance(sentiment, dict):
        sentiment["model_used"] = model_used

    # Self-improvement: save this run to sentiment history for next run's
    intake
    try:
        _save_sentiment_run(
            {
                "timestamp": datetime.now().strftime("%Y-%m-%d
%H:%M:%S"),
                "model_used": model_used,
                "overall_market_sentiment":
sentiment.get("overall_market_sentiment")
                if isinstance(sentiment, dict)
                else "N/A",
                "sentiment_score": sentiment.get("sentiment_score")
                if isinstance(sentiment, dict)
                else None,
                "confidence": sentiment.get("confidence")
                if isinstance(sentiment, dict)
                else "LOW",
            }
        )
    except Exception as hist_err:
        logger.error("Failed to save sentiment history: %s", hist_err)

    _set_persistent_sentiment_cache(sentiment, now, ttl=3600.0)
    return sentiment

if __name__ == "__main__":
    # Setup basic logging to see diagnostic output
    logging.basicConfig(level=logging.INFO)
    # Load env vars for the test
    from dotenv import load_dotenv

    load_dotenv(Path(__file__).resolve().parent.parent / ".env")
    key = os.environ.get("GROQ_API_KEY")
    print(f"DEBUG: GROQ_API_KEY present: {bool(key)}, length: {len(key)
if key else 0}")
    test_llm_providers()

```